

Data Structures Lab – Assignment 4

February 04, 2026

Assignment: Singly Linked Lists Implementation

Objective

To understand the structure and working of a **Singly Linked List** by implementing common linked list operations such as creation, traversal, insertion, deletion, and searching. The lab aims to help students identify the appropriate linked list operation to be used based on a real-world scenario.

Preliminary

A semester project group allows a maximum of **6 students**. Currently, the following **3 students** have confirmed their participation in the given order:

Aditi → Arun → Venu

Create a Singly Linked List named `projectGroup` to store the student names in this order.

Q1. Traversing the Linked List

Write a program to:

- Count the number of students in the project group
- Display the names of all students in the list

Define and call a function named `traverse()` to perform this operation.

Sample Input

Initial Linked List:

Aditi → Arun → Venu

Sample Output

```
Number of students: 3
Students in the project group:
Aditi
Arun
Venu
```

Q2. Searching for a Student in the List

Write a function named `search()` that checks whether a given student name exists in the linked list.

Sample Input 1

Student to search: Venu

Sample Output 1

```
Venu is a member of the project group.
```

Sample Input 2

Student to search: Riya

Sample Output 2

```
Riya is NOT a member of the project group.
```

Q3. Checking Whether the Project Group is Full

The maximum allowed size of the project group is **5 students**.

Write a function named `isFull()` that checks whether more students can be added to the group.

Sample Input

Current group size: 5

Sample Output

```
The project group is full. No more students can be added.
```

Q4. Inserting a Student at the Beginning of the List

The project group decides to appoint a **Group Head**. A new student named **Rohit** is selected as the group head and must be added **at the beginning** of the linked list.

Write a function named `insertStart()` that inserts a new node at the start of the linked list. After insertion, call `traverse()` to display the updated list.

Sample Input

Student to insert at beginning: Rohit

Sample Output

Updated project group:

Rohit -> Aditi -> Arun -> Venu

Q5. Inserting a Student in the Middle of the List

A new student named **Nithin** joins the group. According to alphabetical ordering, the student must be added **after Arun**.

Define a function named `insertMid()` to insert the student at the appropriate position. Call `traverse()` to display the updated list.

Sample Input

Student to insert: Nithin

Insert after: Arun

Sample Output

Updated project group:

Rohit -> Aditi -> Arun -> Nithin -> Venu

Q6. Inserting a Student at the End of the List

Another student named **Yami** joins the project group.

Since the student is added at the end of the list, define a function named `insertEnd()` to perform this operation. Call `traverse()` to display the updated list.

Sample Input

Student to insert: Yami

Sample Output

Updated project group:

Rohit -> Aditi -> Arun -> Nithin -> Venu -> Yami

Q7. Deleting Students from the List

Students **Aditi** and **Arun** decide to leave the project group.

Write a function named `deleteNode()` to delete a given student name from the linked list. Use this function to remove both students and then call `traverse()`.

Sample Input

Students to delete: Aditi, Arun

Sample Output

Updated project group:

Rohit -> Nithin -> Venu -> Yami
